Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: D.Kanishkarane
Reg No: 24011101021

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.
The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.
A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:
- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones
The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:
- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.
Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:
`https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones`
Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training

and test partitions may be used.
Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128\times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32\times 128\times 9}.$$

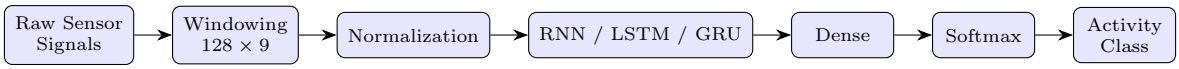
The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:
1. Load the raw inertial signals.
2. Arrange every observation window as 128×9.
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

$$70\% \text{ training}, \quad 15\% \text{ validation}, \quad 15\% \text{ testing}.$$

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

```
Input tensor shape:
Training   : (N_train, 128, 9)
Validation : (N_val,   128, 9)
Testing    : (N_test,  128, 9)
Number of classes: 6
Number of features per time step: 9
Sequence length: 128
```

The actual values of N_{train}, N_{val} and N_{test} depend on the selected subset.

Observed Output

```
Input tensor shape:
Training   : (1680, 128, 9)
Validation : (360, 128, 9)
```

```
Testing    : (360, 128, 9)
Number of classes: 6
Number of features per time step: 9
Sequence length: 128

Train mean after normalisation (should be ~0): [-0.  0. -0. -0.  0.  0.  0. -0.  0.]
Train std  after normalisation (should be ~1): [1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

Class Distribution

Table 1: Class distribution across the training, validation and test splits.

Activity	Train	Validation	Test
WALKING	280	60	60
WALKING_UPSTAIRS	280	60	60
WALKING_DOWNSTAIRS	280	60	60
SITTING	280	60	60
STANDING	280	60	60
LAYING	280	60	60
Total	**1680**	**360**	**360**

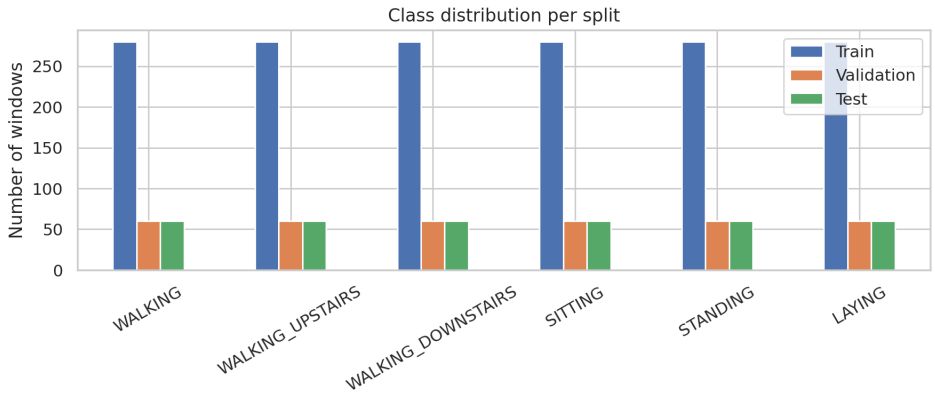


Figure 1: Class distribution per split.

Inference:
The plot shows the number of windows per activity in the training, validation and test sets. Every class has exactly 280 training windows and 60 windows each in validation and testing, so the 2400-window subset (400 per class) was split 70/15/15 with all six classes equally represented in every split. This is expected because the subset was balanced first and the split was stratified. As a result, accuracy is not inflated by a majority class, and a class that performs badly cannot be hidden by class size. The normalised training data has per-channel mean of about 0 and standard deviation of 1. The statistics were computed from the training set only and then applied to the validation and test sets, so no test information leaked into preprocessing.

6. Temporal Data Visualization

Plot 1: Sensor Signal versus Time

One representative window from each of the six activity classes was plotted for three channels: `body_acc_x`, `body_gyro_x` and `total_acc_x`. The raw signal (before normalisation) is shown so that the values keep their physical meaning.

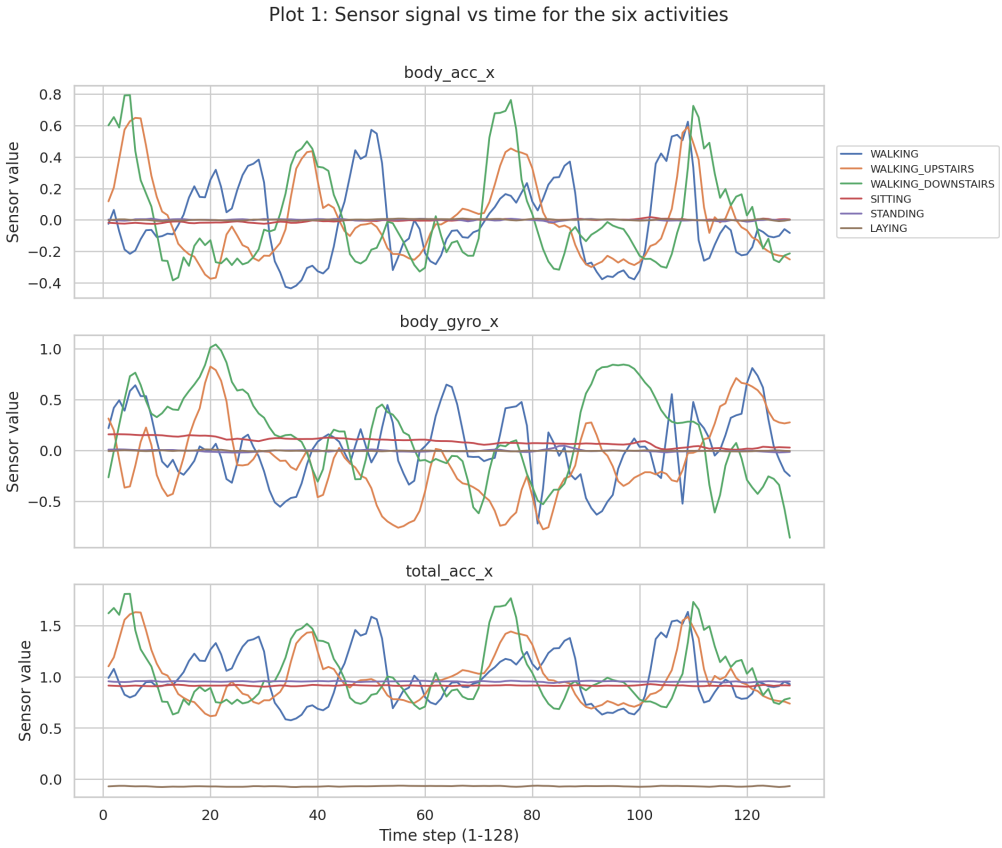


Figure 2: Plot 1: Sensor signal versus time step (1–128) for the six activities.

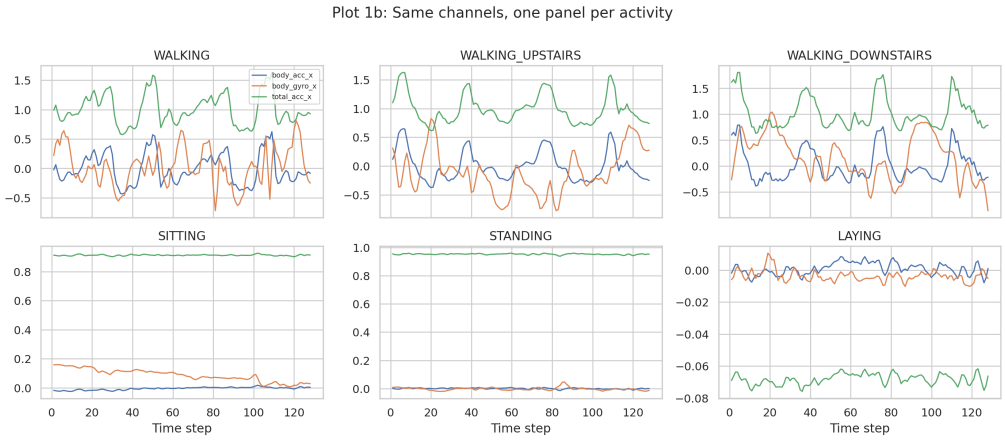


Figure 3: Plot 1b: The same three channels, one panel per activity.

Table 2: Mean within-window standard deviation of the plotted channels for each activity (training windows).

Activity	**std(body_acc_x)**	**std(body_gyro_x)**	**std(total_acc_x)**	**Motion energy**
WALKING	0.2235	0.4681	0.2231	0.1769
WALKING_UPSTAIRS	0.2527	0.4512	0.2518	0.1932
WALKING_DOWNSTAIRS	0.3689	0.5880	0.3682	0.2351
SITTING	0.0069	0.0188	0.0055	0.0120
STANDING	0.0060	0.0426	0.0054	0.0103
LAYING	0.0128	0.0239	0.0083	0.0111

Motion energy is the mean standard deviation of the three body-acceleration axes.

Inference:

1. Temporal pattern. The three walking activities show repeated oscillations across the 128 steps. In `body_acc_x`

the peaks recur roughly every 35 time steps and the values swing between about -0.4 and 0.8, while `body_gyro_x` swings by about ± 0.5 to 1.0. `total_acc_x` oscillates around roughly 1 for the walking activities. SITTING, STANDING and LAYING are almost flat lines. Their body-acceleration standard deviation is between 0.006 and 0.013, compared with 0.22 to 0.37 for the walking activities, which is more than an order of magnitude smaller. For the static activities the only difference is the constant level of `total_acc_x`: about 0.91 for SITTING, about 0.95 for STANDING and about -0.07 for LAYING.

2. Similar activities. WALKING and WALKING_UPSTAIRS look most alike: both are periodic with a similar amplitude range, and they have the smallest distance between activity statistics (0.64). WALKING_DOWNSTAIRS has the highest motion energy (0.2351) and the largest gyroscope variation (0.5880). SITTING and STANDING are almost indistinguishable in the plots (distance 1.11) and differ only by a small offset in `total_acc_x`. LAYING is clearly separated because its `total_acc_x` is near zero rather than near 0.9. The walking variants and the SITTING/STANDING pair are therefore the ones to watch for confusion later.

3. Why ordering matters. The information that separates the walking activities is the rhythm: where the peaks occur, how far apart they are and in which direction the signal moves. Shuffling the 128 readings would keep exactly the same values (same mean and standard deviation) but would destroy this rhythm, so a window of walking would look like random noise. A model that treats the 1152 numbers as unrelated features ignores this structure, whereas a recurrent network reads the steps in order and carries a hidden state forward. The plots therefore motivate sequence models such as RNN, LSTM and GRU for this task.

7. Vanilla RNN Architecture

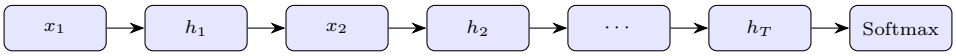
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the whole chain of hidden states $h_1, h_2, \ldots, h_T$. BPTT propagates the error backward through these steps, and since the weights are shared, the weight gradient is the sum of the contributions from all time steps. The gradient reaching an early step is a product of many Jacobians:

$$\frac{\partial h_T}{\partial h_{T-k}} = \prod_{j=T-k+1}^{T} \frac{\partial h_j}{\partial h_{j-1}}.$$

If the factors are below 1 the gradient vanishes, and if they are above 1 it explodes.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \to 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \qquad x_2 = 0.7, \qquad x_3 = 0.2,$$

$$h_0 = 0, \qquad W_x = 0.5, \qquad W_h = 0.8, \qquad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1, h_2 and h_3 manually.
Students must compare their calculated values with the values obtained from a program.

Manual calculation:

$$a_1 = 0.5(0.5) + 0.8(0) + 0.1 = 0.35, \qquad h_1 = \tanh(0.35) \approx 0.3364$$
$$a_2 = 0.5(0.7) + 0.8(0.3364) + 0.1 \approx 0.7191, \qquad h_2 = \tanh(0.7191) \approx 0.6164$$
$$a_3 = 0.5(0.2) + 0.8(0.6164) + 0.1 \approx 0.6931, \qquad h_3 = \tanh(0.6931) \approx 0.6000$$

Table 3: Hand calculation compared with the program (NumPy loop and Keras `SimpleRNN`).

Step	Hand calc	NumPy loop	Keras SimpleRNN	|Hand − Keras|
h_1	0.3364	0.3364	0.3364	2.45×10^{-5}
h_2	0.6164	0.6164	0.6164	4.82×10^{-5}
h_3	0.6000	0.6000	0.6000	4.21×10^{-5}

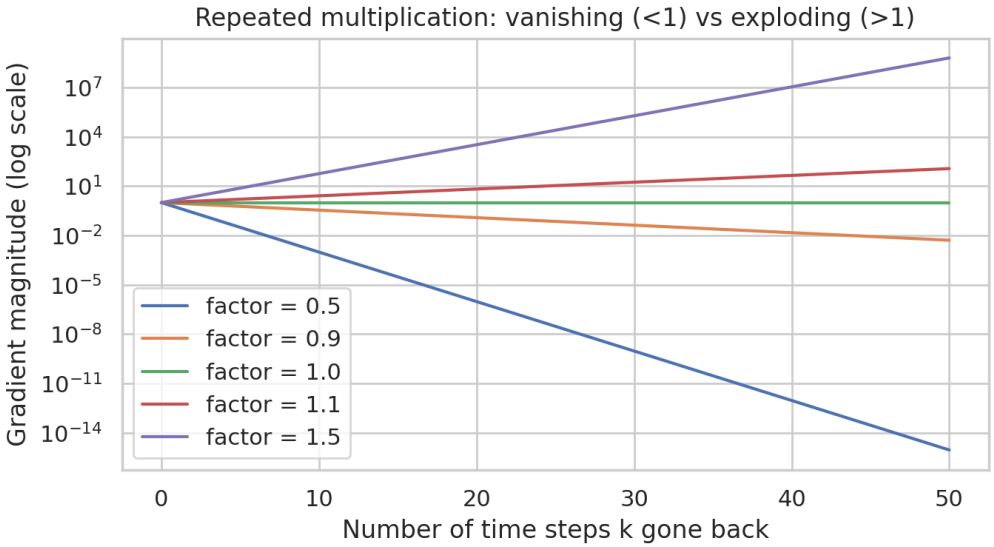
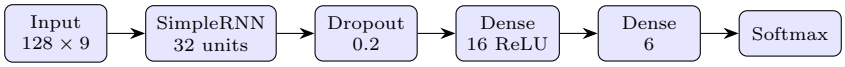


Figure 4: Repeated multiplication of a factor over k time steps: vanishing (< 1) versus exploding (> 1).

Inference:
The hand-calculated hidden states match the NumPy loop and the Keras `SimpleRNN` to four decimal places; the largest difference is about 4.8×10^{-5}, which comes from rounding in the hand calculation and float32 arithmetic in Keras. The plot shows why BPTT is difficult: after 50 steps a factor of 0.5 shrinks the gradient to 8.88×10^{-16} (vanishing), while a factor of 1.5 grows it to 6.38×10^{8} (exploding), and a factor of exactly 1.0 stays constant. Because the gradient to early time steps is a product of many such factors, even a small deviation from 1 becomes extreme over a long sequence. This is why a vanilla RNN struggles with long-term dependencies and why gated models such as LSTM and GRU are introduced.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

Model Summary

Table 4: RNN model summary.

Layer (type)	**Output Shape**	**Param #**
InputLayer	(None, 128, 9)	0
SimpleRNN (32 units)	(None, 32)	1,344
Dropout (0.2)	(None, 32)	0
Dense (16, ReLU)	(None, 16)	528
Dense (6, Softmax)	(None, 6)	102
Trainable parameters		**1,974**

Training Log

Table 5: RNN training progress (selected epochs).

Epoch	**Train Loss**	**Train Acc**	**Val Loss**	**Val Acc**
1	1.6579	0.3607	1.5055	0.4556
5	1.0665	0.5887	0.9851	0.6028
10	0.7365	0.6756	0.7223	0.6500
15	0.6321	0.7262	0.5895	0.7278
20	0.5518	0.7750	0.5276	0.7639
25	0.4364	0.8262	0.4277	0.8111
30	0.3900	0.8542	0.4255	0.8111

Training time was 24.4 s and the test accuracy was 84.17%.
Inference:
The model has only 1,974 trainable parameters: the SimpleRNN layer holds 1,344 of them ($9 \times 32 + 32 \times 32 + 32$), and the two Dense layers add 528 and 102. Over 30 epochs the training accuracy rose from 36.07% to 85.42% and the validation accuracy from 45.56% to 81.11%, while both losses fell steadily (validation loss 1.5055 to 0.4255). Training and validation stay close throughout (final accuracy gap 4.31 points), so there is no sign of strong overfitting. The losses were still falling near the end, so the RNN learns comparatively slowly; the full curves in Plots 2 and 3 will confirm this. The final test accuracy of 84.17% is consistent with the validation accuracy.

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.
The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

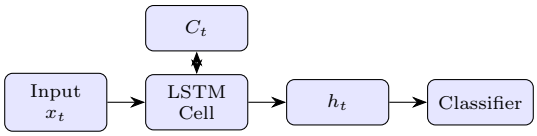
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences. **Required inference:**

The cell state C_t is a separate memory line that is updated by addition, $C_t = f_t C_{t-1} + i_t \tilde{C}_t$. The forget gate f_t (between 0 and 1) scales the old memory: a value near 1 keeps it and a value near 0 erases it. The input gate i_t decides how much of the new candidate $\tilde{C}_t$ is written, and the output gate o_t decides how much of the memory is exposed as h_t. Because the memory is carried forward by an element-wise multiplication with f_t rather than being repeatedly squashed through W_h and tanh, the gradient can flow back along C_t over many steps without shrinking much when $f_t \approx 1$. This lets the LSTM retain useful information over long sequences and discard what is irrelevant.

Model Summary

Table 6: LSTM model summary.

Layer (type)	**Output Shape**	**Param #**
InputLayer	(None, 128, 9)	0
LSTM (32 units)	(None, 32)	5,376
Dropout (0.2)	(None, 32)	0
Dense (16, ReLU)	(None, 16)	528
Dense (6, Softmax)	(None, 6)	102
Trainable parameters		**6,006**

Training Log

Table 7: LSTM training progress (selected epochs).

Epoch	**Train Loss**	**Train Acc**	**Val Loss**	**Val Acc**
1	1.6405	0.3655	1.4591	0.3722
5	0.7929	0.6929	0.7370	0.7167
10	0.5922	0.7756	0.4499	0.8250
15	0.3478	0.8530	0.3522	0.8722
20	0.3226	0.8774	0.3195	0.8861
25	0.2216	0.9208	0.2211	0.9194
30	0.1732	0.9470	0.1959	0.9278

Training time was 22.8 s and the test accuracy was 93.06%.
Inference:
The LSTM has 6,006 trainable parameters, about three times the RNN's 1,974, because its layer has four weight blocks (forget, input, output and candidate) instead of one ($4 \times (9 \times 32 + 32 \times 32 + 32) = 5{,}376$). Over 30 epochs the training accuracy rose from 36.55% to 94.70% and the validation accuracy from 37.22% to 92.78%, with validation loss falling from 1.4591 to 0.1959. The training and validation curves stay close (final accuracy gap 1.92 points), so there is no sign of overfitting. The validation loss was still decreasing at epoch 30, so the model had not fully converged. Compared with the RNN (81.11% validation and 84.17% test accuracy), the LSTM reaches 92.78% validation and 93.06% test accuracy in a similar training time (22.8 s versus 24.4 s), which is consistent with the gated cell state handling the 128-step sequences better than a plain tanh state.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

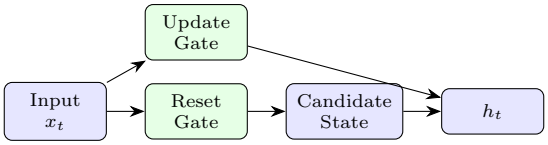
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

Comparison of Gates and State Representations

Table 8: Gates and state representations in RNN, LSTM and GRU.

Property	**RNN**	**LSTM**	**GRU**
Number of gates	0	3 (forget, input, output)	2 (update, reset)
Hidden state h_t	Yes	Yes	Yes
Separate cell state C_t	No	Yes	No
Weight blocks per layer	1	4	3

The RNN keeps only a hidden state. The LSTM adds a cell state and three gates. The GRU uses two gates and no separate cell state: its update gate z_t plays the role of the LSTM's forget and input gates combined, and h_t acts as both memory and output.

Model Summary

Table 9: GRU model summary.

Layer (type)	**Output Shape**	**Param #**
InputLayer	(None, 128, 9)	0
GRU (32 units)	(None, 32)	4,128
Dropout (0.2)	(None, 32)	0
Dense (16, ReLU)	(None, 16)	528
Dense (6, Softmax)	(None, 6)	102
Trainable parameters		**4,758**

Training Log

Table 10: GRU training progress (selected epochs).

Epoch	**Train Loss**	**Train Acc**	**Val Loss**	**Val Acc**
1	1.6532	0.3071	1.4949	0.4194
5	0.7703	0.7000	0.6721	0.7333
10	0.3178	0.8964	0.3400	0.8917
15	0.1592	0.9548	0.2180	0.9250
20	0.1441	0.9542	0.1522	0.9472
25	0.1079	0.9649	0.1469	0.9444
30	0.1003	0.9661	0.1405	0.9500

Training time was 20.3 s and the test accuracy was 95.56%.

Inference:

The GRU has 4,758 trainable parameters, about 79% of the LSTM's 6,006, because it has three weight blocks instead of four ($3 \times (9 \times 32 + 32 \times 32 + 2 \times 32) = 4{,}128$ in the recurrent layer). It also learned fastest: the validation accuracy already reached 89.17% at epoch 10 (LSTM 82.50%, RNN 65.00%) and ended at 95.00%, with validation loss falling from 1.4949 to 0.1405. The training and validation curves stay close (final accuracy gap 1.61 points), and the validation loss changed little between epochs 25 and 30 (0.1469 to 0.1405), so there is no overfitting and the model is close to converged. With the shortest training time (20.3 s) and the highest test accuracy (95.56%, versus 93.06% for the LSTM and 84.17% for the RNN), the GRU gave the best result of the three models in

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

Controlled Comparison Setup

Table 11: Experimental conditions shared by RNN, LSTM and GRU.

Setting	**RNN**	**LSTM**	**GRU**
Input shape	128×9	128×9	128×9
Recurrent units	32	32	32
Dropout / Dense head	0.2 / 16 ReLU, 6 Softmax	0.2 / 16 ReLU, 6 Softmax	0.2 / 16 ReLU, 6 Softmax
Optimizer, learning rate	Adam, 10^{-3}	Adam, 10^{-3}	Adam, 10^{-3}
Batch size, epochs	32, 30	32, 30	32, 30
Loss	Sparse CCE	Sparse CCE	Sparse CCE

Table 12: Trainable parameters by component.

Component	**RNN**	**LSTM**	**GRU**
Recurrent layer	1,344	5,376	4,128
Dense (16) + Dense (6)	630	630	630
Total trainable	**1,974**	**6,006**	**4,758**

Inference:
Only the recurrent layer differs between the three models; the input, dropout, dense head, optimizer, batch size, epochs and loss are identical, so any difference in results comes from the recurrent cell itself. The Dense head has the same 630 parameters in every model ($32 \times 16 + 16 + 16 \times 6 + 6$), so the entire difference in size comes from the recurrent layer. It grows with the number of weight blocks: 1 block for the RNN ($9 \times 32 + 32 \times 32 + 32 = 1{,}344$), 4 for the LSTM (four times that, 5,376) and 3 for the GRU (4,128, using Keras' two bias vectors per gate). The GRU is therefore about 79% of the LSTM's total size, and the LSTM is about three times the RNN's.

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:
Plot 2: Training and Validation Loss
- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

Plot 3: Training and Validation Accuracy
- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

Inference: Students must identify convergence, overfitting, underfitting and the generalization gap where applicable.

Training Curves

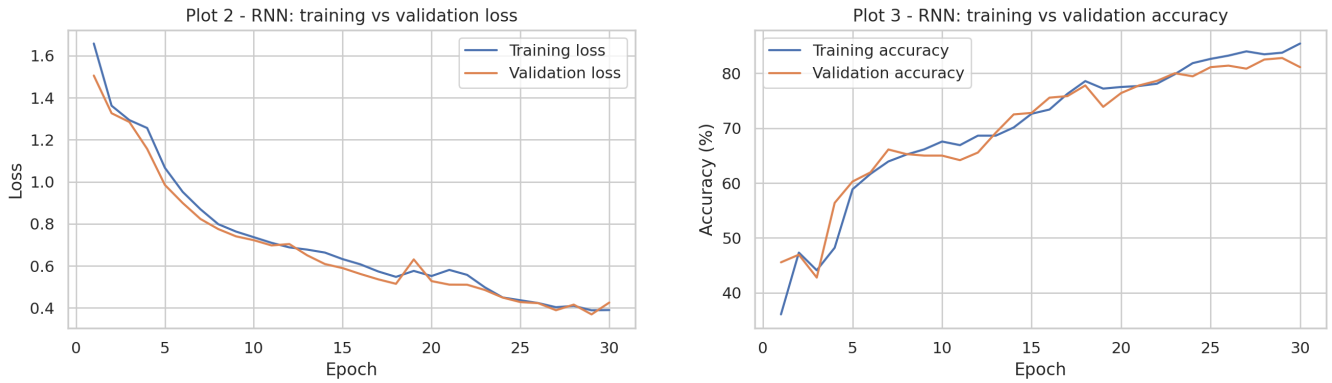


Figure 5: RNN: training vs validation loss (Plot 2, left) and accuracy (Plot 3, right).

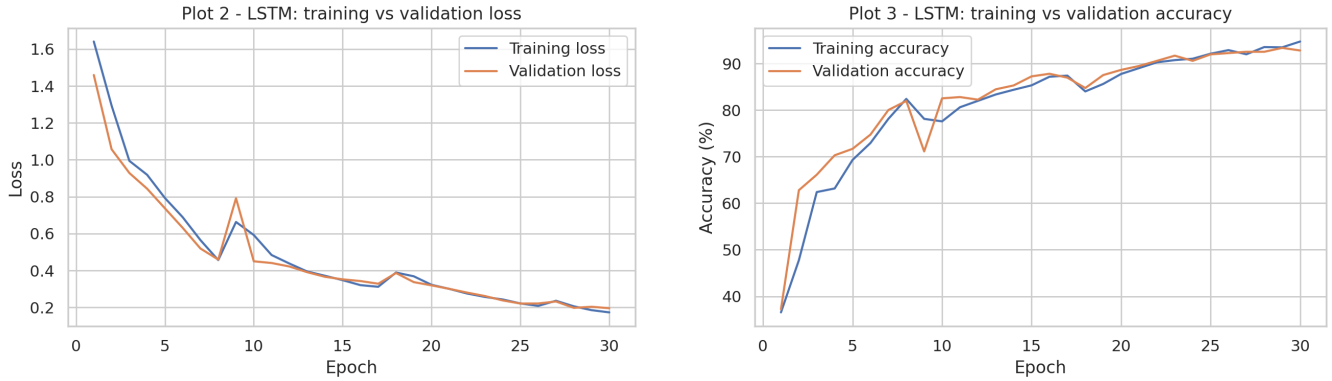


Figure 6: LSTM: training vs validation loss (Plot 2, left) and accuracy (Plot 3, right).

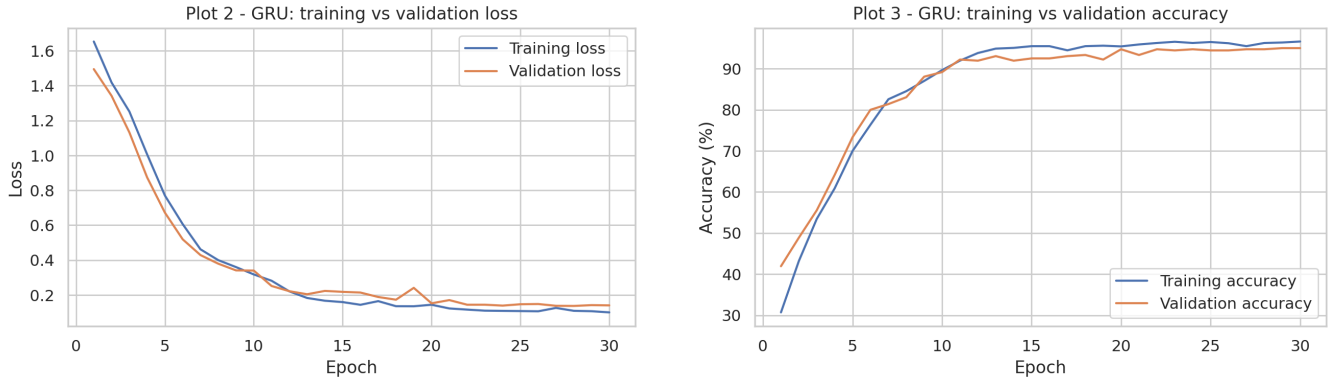


Figure 7: GRU: training vs validation loss (Plot 2, left) and accuracy (Plot 3, right).

Table 13: Convergence and generalisation summary (30 epochs).

Quantity	**RNN**	**LSTM**	**GRU**
Lowest validation loss (epoch)	0.3693 (29)	0.1959 (30)	0.1373 (28)
Final training / validation loss	0.3900 / 0.4255	0.1732 / 0.1959	0.1003 / 0.1405
Final training / validation accuracy (%)	85.42 / 81.11	94.70 / 92.78	96.61 / 95.00
Generalisation gap (points)	4.31	1.92	1.61
First epoch with validation accuracy $\geq 90\%$	never	22	11

Convergence Inferences

Inference (RNN):
Both losses fall steadily from about 1.66 to 0.39 and the accuracy rises from 36% to 85% (training) and 81% (validation), with the two curves close together. The curves are still improving at epoch 30, so the RNN has not fully converged, and validation accuracy never reaches 90% (best 82.8%). The rise in validation loss from 0.3693 to 0.4255 in the last epoch and the small spike near epoch 19 are one-epoch fluctuations, not a trend. The slow, noisy progress is consistent with a vanilla RNN squeezing 128 steps through a single tanh state, where gradients shrink during BPTT.

Inference (LSTM):
The LSTM learns faster than the RNN: validation accuracy passes 90% at epoch 22 and ends at 92.78% (training 94.70%), with validation loss falling to 0.1959. There is one clear instability at epoch 9, where the validation loss jumps to about 0.79 and the validation accuracy drops to about 71%, followed by a smaller bump near epoch 18; training recovers immediately each time. The lowest validation loss occurs at the last epoch, so the LSTM was still improving slowly and would probably gain from more epochs.

Inference (GRU):
The GRU converges fastest: validation accuracy reaches 90% at epoch 11, and after about epoch 12 both curves flatten, ending at 95.00% validation accuracy and 0.1405 validation loss. The lowest validation loss (0.1373) is at epoch 28, only 2.4% below the final value, and a small spike near epoch 19 recovers within a few epochs. The GRU can therefore be regarded as converged within the 30 epochs. Its quick, stable learning is consistent with the update gate providing a short gradient path through time with fewer parameters than the LSTM.

Inference (overfitting, underfitting and generalisation gap):
None of the three models overfits: training and validation curves stay close and the validation loss never rises steadily while the training loss falls. The generalisation gap (training minus validation accuracy) is 4.31 points for the RNN, 1.92 for the LSTM and 1.61 for the GRU. The RNN shows the largest gap and the weakest fit (training accuracy only 85.4% and still rising), so it is the model closest to underfitting, limited by how quickly it can learn rather than by memorisation. The LSTM and GRU fit the training data much better with a small gap, which means they generalise well on this task.

14. Performance Evaluation

Evaluate each model on the independent test set.
The following metrics are mandatory:
- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	84.17	93.06	95.56
Macro Precision (%)	84.52	93.21	95.78
Macro Recall (%)	84.17	93.06	95.56
Macro F1 (%)	84.23	93.02	95.54
Parameters	1974	6006	4758
Training Time (s)	24.43	22.85	20.28

Inference:
The GRU gives the best test result (95.56% accuracy, 95.54% macro F1), followed by the LSTM (93.06%, 93.02%) and the RNN (84.17%, 84.23%), the same order as on the validation set. Macro recall equals accuracy for every model because the test set is balanced (60 windows per class). The RNN trails the gated models by about 9 to 11 points, mainly on WALKING and WALKING_UPSTAIRS (F1 of 72.44% and 69.57%). LAYING is classified perfectly by all three models. The LSTM and GRU errors are concentrated in SITTING and STANDING (F1 between 85.7% and 87.5%), and the GRU classifies all three walking activities perfectly. The GRU also has fewer parameters than the LSTM (4,758 versus 6,006) and trained fastest (20.28 s), so it was best on every measure in this run. With only 360 test windows, one window is about 0.28 points, so the 2.5-point gap between GRU and LSTM is modest.

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix
Generate a separate confusion matrix for each model.
Students must identify:
- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

The inference must be based on the student's observed matrix rather than on an assumed result.

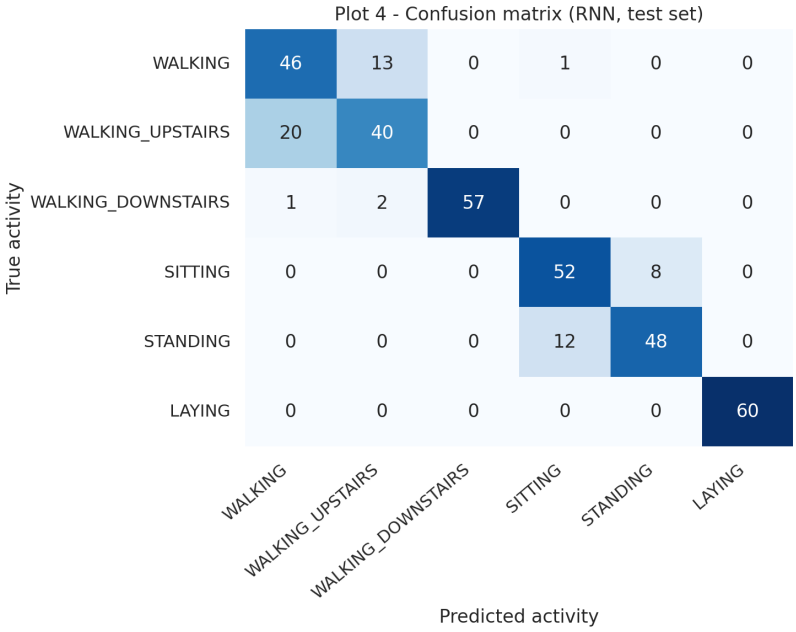


Figure 8: Plot 4: Confusion matrix of the RNN on the test set.

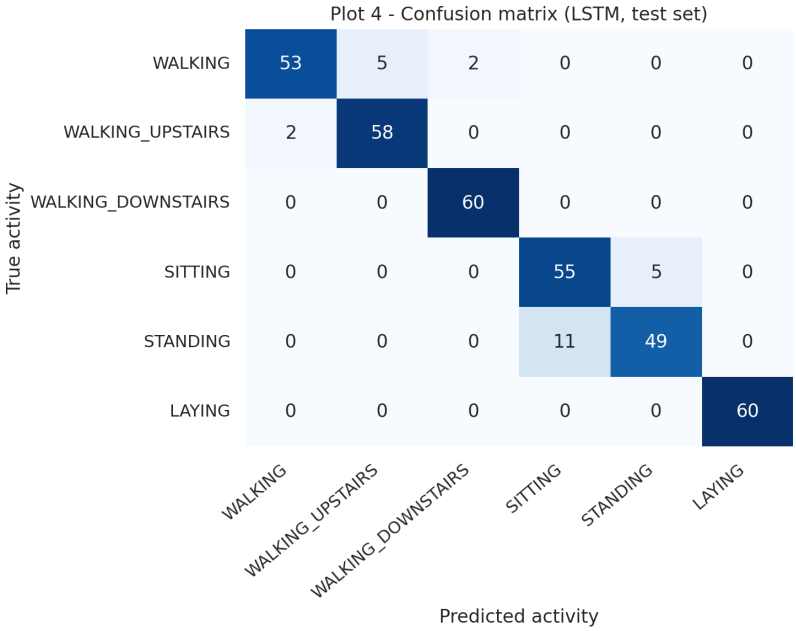


Figure 9: Plot 4: Confusion matrix of the LSTM on the test set.

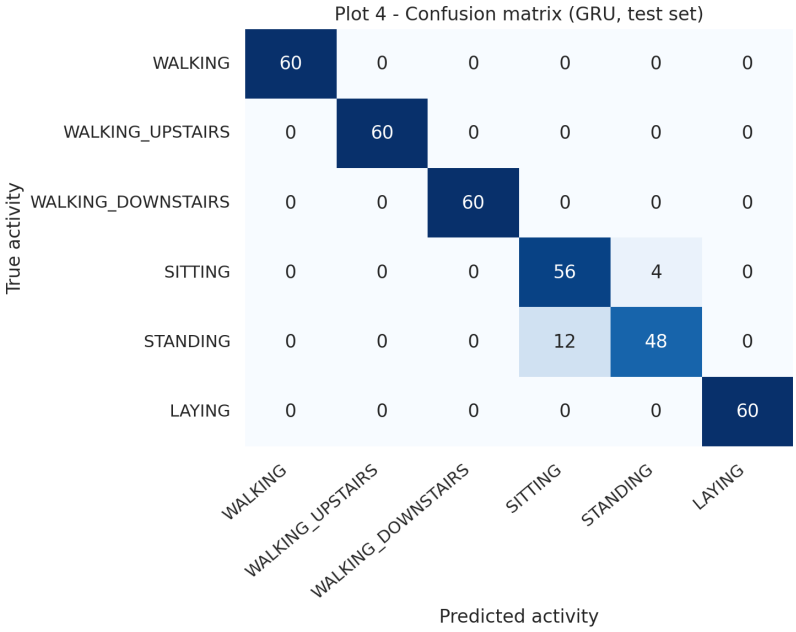


Figure 10: Plot 4: Confusion matrix of the GRU on the test set.

Table 14: Per-class recall (%) with the number of misclassified windows in brackets (60 test windows per class).

Activity	**RNN**	**LSTM**	**GRU**
WALKING	76.67 (14)	88.33 (7)	100.00 (0)
WALKING_UPSTAIRS	66.67 (20)	96.67 (2)	100.00 (0)
WALKING_DOWNSTAIRS	95.00 (3)	100.00 (0)	100.00 (0)
SITTING	86.67 (8)	91.67 (5)	93.33 (4)
STANDING	80.00 (12)	81.67 (11)	80.00 (12)
LAYING	100.00 (0)	100.00 (0)	100.00 (0)
Total errors	**57**	**25**	**16**

Table 15: Most confused activity pairs (errors in both directions combined).

Pair	**RNN**	**LSTM**	**GRU**
WALKING ↔ WALKING_UPSTAIRS	33 (13 + 20)	7 (5 + 2)	0
SITTING ↔ STANDING	20 (8 + 12)	16 (5 + 11)	16 (4 + 12)

Inference:

Highest recognition rate. LAYING is recognised perfectly (60/60) by all three models. WALKING_DOWNSTAIRS is also perfect for the LSTM and GRU, and the GRU additionally classifies WALKING and WALKING_UPSTAIRS perfectly.

Largest number of misclassifications. For the RNN it is WALKING_UPSTAIRS (20 of 60 misclassified, recall 66.67%), whereas for both the LSTM (11 errors) and the GRU (12 errors) it is STANDING (recall 81.67% and 80.00%).

Frequently confused pairs. The RNN's biggest confusion is WALKING ↔ WALKING_UPSTAIRS (33 errors), followed by SITTING ↔ STANDING (20). The LSTM confuses SITTING and STANDING most (16), with only 7 walking/upstairs errors, and all 16 GRU errors are SITTING ↔ STANDING. In every model the STANDING-predicted-as-SITTING error is larger than the reverse (12 versus 8 for the RNN, 11 versus 5 for the LSTM and 12 versus 4 for the GRU).

Consistency across models. The errors are only partly consistent. SITTING ↔ STANDING is the one confusion that appears in all three models with a similar size (20, 16 and 16), and STANDING stays at about 80% recall regardless of the architecture. This points to a difficulty in the data: both are static postures with almost identical body-acceleration and gyroscope signals (as seen in Plot 1), so extra recurrent capacity does not help much. The WALKING ↔ WALKING_UPSTAIRS confusion, on the other hand, shrinks from 33 errors (RNN) to 7 (LSTM) and 0 (GRU), so it depends on the model and is mostly a limitation of the plain RNN, which struggles to separate

two similar periodic gaits. Accuracy alone would hide this: the matrices show that the gated models' remaining errors are almost all one pair of static postures.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time	24.43 s	22.85 s	20.28 s
Test F1-score	84.23%	93.02%	95.54%

Plot 5: Model Performance Comparison
Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.

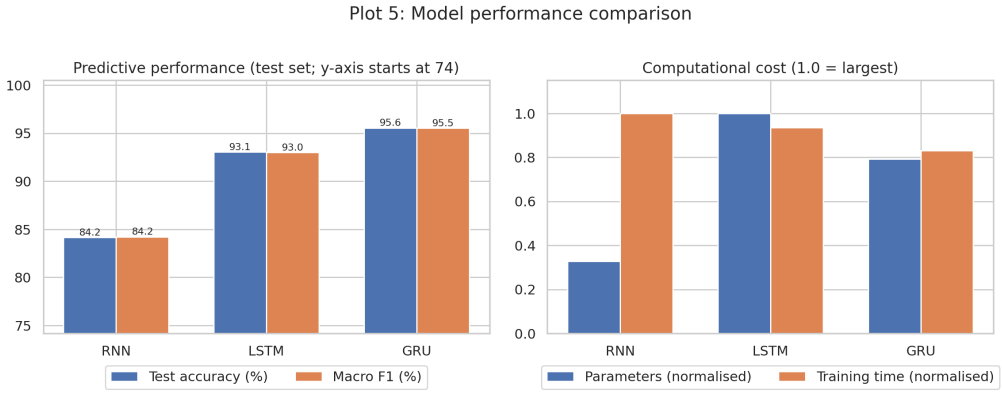


Figure 11: Plot 5: Test accuracy and macro F1 (left, y-axis starts at 74) and normalised parameters and training time (right, 1.0 = largest).

Required inference:
Students should discuss the trade-off among:

$$\text{Predictive Performance} \quad \leftrightarrow \quad \text{Model Complexity} \quad \leftrightarrow \quad \text{Training Cost.}$$

Inference:
The left plot shows performance rising from the RNN (84.2% accuracy, 84.2% F1) to the LSTM (93.1%, 93.0%) and the GRU (95.6%, 95.5%). The right plot shows that model size follows a different order: the LSTM is the largest (6,006 parameters, 3.04 times the RNN's 1,974), the GRU is in between (4,758, 2.41 times the RNN and 79% of the LSTM) and the RNN is the smallest. This follows from the number of weight blocks per layer (one for the RNN, four for the LSTM, three for the GRU). Training time does not follow model size: the RNN was the slowest (24.43 s) despite having the fewest parameters, the LSTM took 22.85 s (0.94 times the RNN) and the GRU 20.28 s (0.83 times). A likely reason is that the LSTM and GRU use optimised GPU kernels while `SimpleRNN` does not, so fewer parameters does not guarantee faster training.

Trade-off. Going from the RNN to a gated model costs about 2.4 to 3 times the parameters, but it buys 8.8 F1 points (LSTM) or 11.3 F1 points (GRU) and did not increase training time here. Between the two gated models, the GRU is better on all three axes in this run: higher F1 (95.54% versus 93.02%), 21% fewer parameters and the shortest training time. So for this task the simplest model is not the cheapest in time, and the largest model is not the best; the GRU gives the best balance of performance, complexity and cost. The 2.5-point gap between GRU and LSTM is about 9 of the 360 test windows and comes from a single run, so it should be read as an indication rather than a firm ranking.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	74.82	92.46	91.39
64	71.02	93.90	93.32
128	84.23	93.02	95.54

Values are test macro F1 (%). Each shorter input was built by keeping the first T time steps of every window, and the same split, normalisation procedure, models and training settings were used for all lengths.

Plot 6: Sequence Length vs. Test F1-score

Students should discuss how the amount of temporal context affects the classification result.

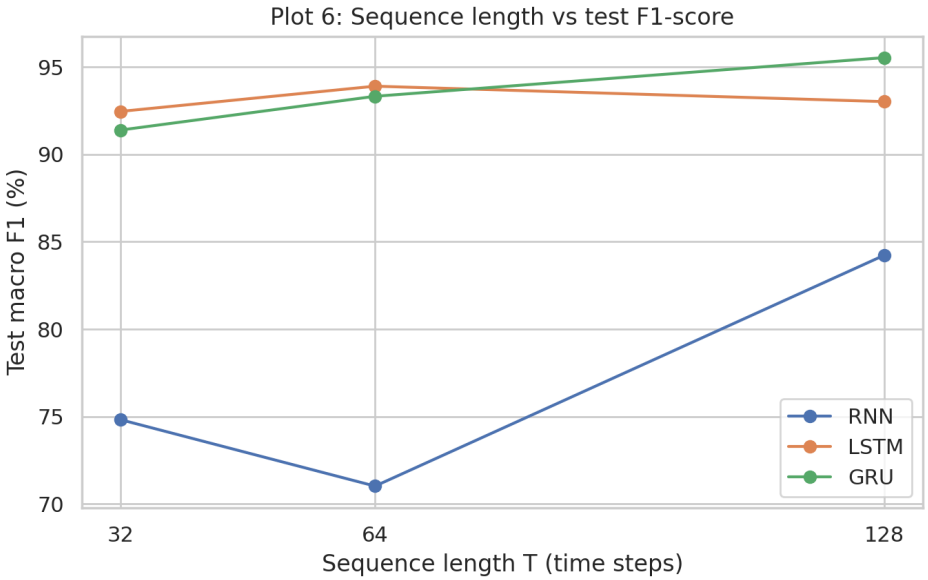


Figure 12: Plot 6: Test macro F1-score for RNN, LSTM and GRU at $T = 32, 64, 128$.

Inference:

The plot shows the test macro F1-score of each model as more of the window is used (at 50 Hz, $T = 32$, 64 and 128 correspond to 0.64 s, 1.28 s and 2.56 s). The GRU improves steadily with context, from 91.39% to 93.32% to 95.54% (+4.15 points). The RNN gains the most overall (74.82% to 84.23%, +9.41 points) but not smoothly: it drops to 71.02% at $T = 64$ before rising at $T = 128$. The LSTM is almost flat, 92.46%, 93.90% and 93.02% (+0.56 points overall), and its best result is at $T = 64$. So more temporal context helps clearly for the RNN and GRU, but not for the LSTM in this run.

A likely reason is that short windows hold less of the repeated movement pattern, so similar activities are harder to separate. The plain RNN depends on this the most (74.82% at $T = 32$ versus 84.23% at $T = 128$), while the gated models already reach above 91% with only 32 steps. The RNN dip at $T = 64$ and the LSTM drop from 64 to 128 (0.88 points, about 3 of the 360 test windows) are small and may just be run-to-run variation from a single training run. Training time rose about 1.4 to 1.5 times from $T = 32$ to $T = 128$ (RNN 17.05 s to 24.43 s, GRU 14.85 s to 20.28 s). Overall, more temporal context improves or at least maintains F1, and the best test F1 of the study (95.54%) is the GRU with the full 128 steps.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:
`https://www.crcv.ucf.edu/data/UCF101.php`
Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.
Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.
Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.
If the CNN produces a feature vector of dimension D, then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.
For example:

```
Predicted class : Basketball
Actual class    : Basketball
Confidence      : 0.xx
```

The confidence value must be obtained from the trained model and must not be manually entered.

Table 16: Number of videos per class in each split.

Split	Basketball	Biking	TennisSwing	WalkingWithDog	Total
Train	40	40	40	40	160
Validation	10	10	10	10	40
Test	12	12	12	12	48

Inference:
The subset has 248 videos in total, with exactly the same number of videos per class within each split, so the classes are balanced and accuracy will not be dominated by one action. The videos were split by UCF101 clip group (groups 1–4 for testing, 5–7 for validation and 8–25 for training), because clips from the same group come from one long recording and a random split would put near-duplicate clips in both training and test sets. The test set is small (48 videos, so one video is about 2.1 percentage points), and the results in the next sections should be read with that in mind. The predicted class and confidence for each test video are produced by the trained model (Section 21) and are not entered by hand.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

Observed Output

```
MobileNetV2 (frozen): 2,257,984 parameters, all non-trainable.
CNN feature dimension D = 1280
Tensor supplied to the recurrent network: (B, 10, 1280)
  train: X = (160, 10, 1280), y = (160,)
  val  : X = (40, 10, 1280),  y = (40,)
  test : X = (48, 10, 1280),  y = (48,)
```

Table 17: Feature tensors supplied to the recurrent network.

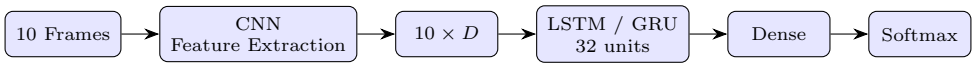
Split	**Feature tensor X**	**Labels y**
Training	(160, 10, 1280)	(160,)
Validation	(40, 10, 1280)	(40,)
Testing	(48, 10, 1280)	(48,)

Inference:

The pretrained MobileNetV2 (ImageNet weights, classification head removed, global average pooling) was frozen, so all of its 2,257,984 parameters are non-trainable and only the recurrent classifier is trained. Each $224 \times 224 \times 3$ frame (150,528 values) is converted into a feature vector of dimension $D = 1280$, about 118 times fewer numbers, and a video of 10 frames becomes a sequence $X \in \mathbb{R}^{10\times 1280}$. The recurrent network therefore receives a 3-D tensor $(B, 10, 1280)$, in the same (N, T, F) format as the sensor data, with $T = 10$ frames as the time steps and $F = 1280$ CNN features per step. Extracting the features once and reusing them keeps training fast, which matters because there are only 160 training videos.

21. CNN–LSTM / CNN–GRU Model

Use the following architecture:



Model and Selection

The recurrent classifier is `Input`$(10 \times 1280) \rightarrow$ LSTM/GRU (32 units) $\rightarrow$ Dropout (0.3) $\rightarrow$ Dense (4, softmax). The small dropout was added because there are only 160 training videos. The CNN–LSTM has 168,064 parameters in the LSTM layer plus 132 in the Dense layer, 168,196 trainable in total. LSTM and GRU were trained on the same features, and the model was chosen using **validation** results only.

Table 18: CNN–LSTM and CNN–GRU on identical MobileNetV2 features.

Quantity	**CNN–LSTM**	**CNN–GRU**
Best validation accuracy (%)	90.00	90.00
Final validation loss	0.2058	0.2448
Test accuracy (%)	85.42	91.67
Test macro F1 (%)	84.84	91.59
Parameters	168,196	126,276
Training time (s)	7.8	6.6

Both models reached the same best validation accuracy (90.00%), so the CNN–LSTM was selected because of its lower validation loss. The test set was not used for this choice.

Plot 7: Video Sample Frames

Display a representative sequence of sampled frames from one video.



Figure 13: Plot 7: 10 uniformly sampled frames of `v_Basketball_g01_c01.avi`.

Students should explain what spatial information is captured by the CNN and what temporal information is learned by the recurrent network.

Inference (spatial and temporal information):

The frozen CNN looks at one frame at a time and captures spatial information: the indoor court, the hoop, the players and the ball, reduced to a 1280-dimensional vector per frame. The LSTM/GRU reads the 10 vectors in order and learns how they change over time, here the player bending, jumping with raised arms (frame 3) and landing. Only the final hidden state goes to the softmax classifier.

Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture.

Table 19: CNN–LSTM training progress (selected epochs).

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	0.8864	0.7125	0.6234	0.8250
5	0.0916	1.0000	0.3261	0.9000
10	0.0313	1.0000	0.2633	0.9000
15	0.0169	1.0000	0.2328	0.8750
20	0.0130	1.0000	0.2205	0.8750
25	0.0091	1.0000	0.2131	0.8750
30	0.0084	1.0000	0.2058	0.8750

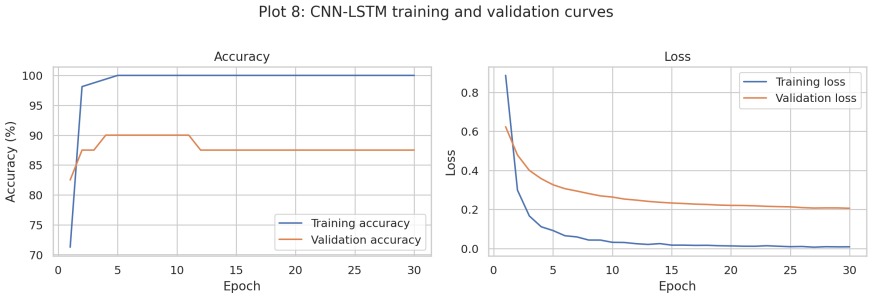


Figure 14: Plot 8: CNN–LSTM training and validation accuracy (left) and loss (right).

Inference:

Training accuracy reaches 100% by epoch 5 and the training loss falls to 0.0084, while validation accuracy peaks at 90% (epochs 4–11) and settles at 87.5%, with the validation loss falling slowly to 0.2058 without rising. The final gap is 12.5 points, a mild overfitting: strong pretrained features let the classifier fit the 160 training videos almost perfectly, but it generalises less well. With 40 validation videos, one video is 2.5 points, so the drop from 90% to 87.5% is a single video.

Plot 9: Video Confusion Matrix

Analyze class-level recognition and temporal confusions.

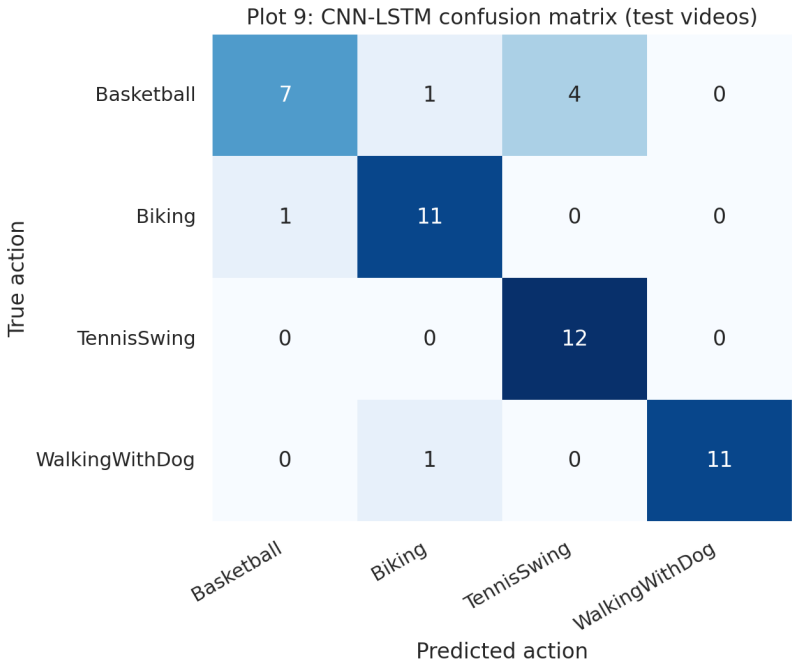


Figure 15: Plot 9: CNN–LSTM confusion matrix on the 48 test videos.

Table 20: Per-class recall of CNN–LSTM on the test videos (12 videos per class).

Action	**Correct / 12**	**Recall (%)**
Basketball	7	58.33
Biking	11	91.67
TennisSwing	12	100.00
WalkingWithDog	11	91.67

Inference:
TennisSwing is recognised best (12/12) and Basketball worst (7/12). Of the 7 errors, 4 are Basketball predicted as TennisSwing, so this is the main confusion, and TennisSwing precision drops to 75% (12 of 16 predictions). These four videos are clips c01, c03, c05 and c07 of the same recording (g01), predicted with high confidence (0.67 to 0.96), which points to a problem with that recording (Plot 7 shows small, distant players) rather than random noise. With 12 test videos per class, one recording moves the Basketball recall by 33 points, so class-level numbers are unstable.

Predicted Class and Confidence

The confidence is the maximum softmax probability produced by the trained model.

```
Video           : v_Basketball_g01_c01.avi
Predicted class : TennisSwing
Actual class    : Basketball
Confidence      : 0.92

Video           : v_Basketball_g01_c03.avi
Predicted class : TennisSwing
Actual class    : Basketball
Confidence      : 0.96

Video           : v_Basketball_g01_c05.avi
Predicted class : TennisSwing
Actual class    : Basketball
Confidence      : 0.91

Video           : v_Basketball_g01_c07.avi
Predicted class : TennisSwing
Actual class    : Basketball
Confidence      : 0.67
```

```
Video           : v_Basketball_g02_c02.avi
Predicted class : Basketball
Actual class    : Basketball
Confidence      : 0.80

Video           : v_Basketball_g02_c04.avi
Predicted class : Basketball
Actual class    : Basketball
Confidence      : 0.58
```

The wrong predictions have higher confidence (0.92, 0.96, 0.91) than the two correct ones (0.80, 0.58), so the softmax confidence should not be read as a guarantee of correctness.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.
For a simple laboratory task, construct a synthetic reversal dataset.
Example:

$$[1, 4, 7, 2] \to [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.
The task is intentionally simple so that the encoder–decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

Dataset and Model

The symbols are the integers 1–9, and every sequence has length 4, so there are $9^4 = 6561$ possible sequences. All of them were generated once and split 70/15/15 without overlap, so no test sequence was seen during training. Token 0 is a special START token for the decoder.

Table 21: Reversal dataset and encoder–decoder configuration.

Item	**Value**
Training / validation / test sequences	4592 / 984 / 985
Vocabulary (START + digits 1–9)	10 tokens
Embedding size	32
Encoder LSTM / Decoder LSTM units	128 / 128
Output layer	Dense (10, softmax)
Optimizer, learning rate	Adam, 10^{-3}
Batch size, epochs	64, 60
Loss	Sparse categorical cross-entropy

Table 22: Trainable parameters of the encoder–decoder model.

Layer	**Param #**
Encoder embedding	320
Decoder embedding	320
Encoder LSTM (128)	82,432
Decoder LSTM (128)	82,432
Decoder output Dense (10)	1,290
Total trainable	**166,794**

Teacher forcing. During training the decoder input is the true target shifted right by one step, for example `[START, 2, 7, 4]` for the target $[2, 7, 4, 1]$. At test time no ground truth is available, so the decoder feeds its own previous prediction back in (greedy decoding).

Training Log

Table 23: Seq2seq training progress (selected epochs, teacher-forced).

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	1.9227	0.2798	1.4110	0.4118
10	0.0070	1.0000	0.0068	1.0000
20	0.0015	1.0000	0.0016	1.0000
30	0.0006	1.0000	0.0007	1.0000
40	0.0003	1.0000	0.0004	1.0000
50	0.0002	1.0000	0.0002	1.0000
60	0.0001	1.0000	0.0001	1.0000

Training time was 42.1 s.

Inference:

The encoder–decoder learns the reversal very quickly: by epoch 10 the loss has fallen from 1.9227 to 0.0070 and the teacher-forced accuracy is already 100%, and it keeps decreasing to 0.0001 at epoch 60. Training and validation losses stay almost identical throughout, so there is no overfitting, which is expected because the validation sequences were never seen in training. These accuracies use the true previous token as decoder input, so they are not the final result. The token and sequence accuracies on the test set with real greedy decoding are reported in Section 24.

23. Expected Sequence-to-Sequence Input and Output

Input example:

$$[3, 8, 1, 5]$$

Expected output:

$$[5, 1, 8, 3].$$

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1	[3, 1, 8, 2]	[2, 8, 1, 3]
2	[3, 7, 4, 4]	[4, 4, 7, 3]
3	[4, 3, 3, 5]	[5, 3, 3, 4]
4	[5, 2, 6, 6]	[6, 6, 2, 5]
5	[2, 3, 9, 6]	[6, 9, 3, 2]

Inference:

All five predictions are exactly the input sequences in reverse order, for example $[3, 1, 8, 2] \to [2, 8, 1, 3]$. The decoder produced them one token at a time from the encoder's context, feeding its own previous output back in. Inputs with repeated symbols (samples 2, 3 and 4) are also reversed correctly, so the model is tracking the position of each symbol and not just which symbols appear. Five examples only illustrate the behaviour, and the accuracy over all 985 test sequences is reported in Section 24.

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

Table 24: Sequence-to-sequence results on the test set (985 sequences, 3940 tokens).

Metric	**Value**
Token accuracy (%)	100.00
Sequence accuracy (%)	100.00
Final training loss	0.0001
Final validation loss	0.0001
Parameters	166,794
Training time (s)	42.11

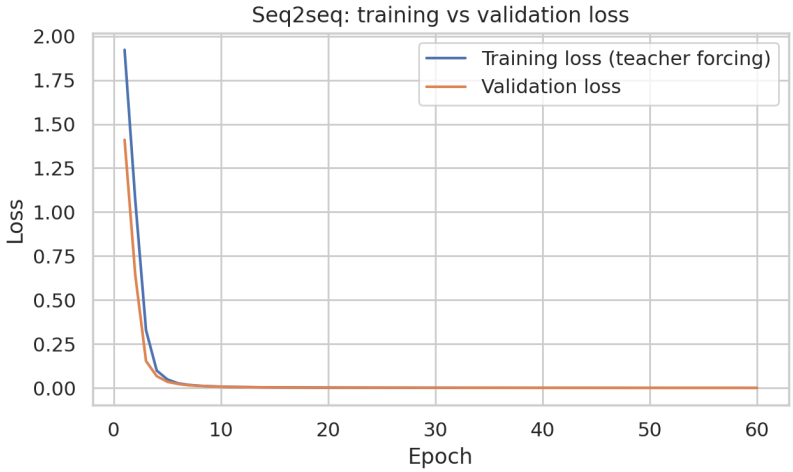


Figure 16: Seq2seq training (teacher forcing) and validation loss over 60 epochs.

Token accuracy by output position was 100% at all four positions. Token and sequence accuracy were measured with real greedy decoding (the decoder uses its own previous prediction), not with teacher forcing.

Inference:

In this run the model is perfect: all 3940 test tokens and all 985 test sequences are correct, and the training and validation losses both end at 0.0001 with no gap, so there is no overfitting. Because there are no token errors, the two accuracies are equal here and this run cannot show the difference between them.

In general, sequence accuracy can be much lower than token accuracy because a sequence counts as correct only if every token is correct, so one wrong token makes the whole sequence wrong. If tokens are wrong independently with token accuracy p, sequence accuracy is about p^L. For our length $L = 4$, a token accuracy of 95% gives only about $0.95^4 \approx 81\%$ sequence accuracy, and 90% gives about $0.90^4 \approx 66\%$. Errors also compound in a decoder, because a wrong token is fed back as input to the next step. The gap therefore grows with sequence length, and sequence accuracy is the stricter and more honest measure for seq2seq tasks.

25. Consolidated Results

Students must complete the following table.

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	84.17	84.52	84.17	84.23	1974
LSTM	93.06	93.21	93.06	93.02	6006
GRU	95.56	95.78	95.56	95.54	4758
CNN–LSTM/GRU	85.42	86.78	85.42	84.84	168196

All values are percentages on the test set, and precision, recall and F1 are macro-averaged. The first three rows are the 6-class activity task (360 test windows). The last row is the 4-class video task (48 test videos, CNN–LSTM selected on validation), and its parameter count is that of the recurrent classifier only, since the frozen MobileNetV2 (2,257,984 parameters) is not trained.

A separate table for the sequence-to-sequence task:

Metric	Reversal task
Token Accuracy (%)	100.00
Sequence Accuracy (%)	100.00
Training Loss	0.0001
Validation Loss	0.0001
Parameters	166794

Accuracies are measured on the 985 test sequences with greedy decoding (Encoder–Decoder LSTM).

26. Required Inferences

Inference:
The three walking activities oscillate periodically (`body_acc_x` peaks about every 35 steps), while SITTING, STANDING and LAYING are almost flat (std 0.006–0.013 versus 0.22–0.37) and differ only in the level of `total_acc_x`. WALKING and WALKING_UPSTAIRS are the most similar (distance 0.64), then SITTING and STANDING (1.11). Order matters because the rhythm exists only in the sequence: shuffling the 128 readings keeps the values but destroys the pattern.
Inference:
The hand-calculated h_1, h_2, h_3 match the NumPy loop and Keras `SimpleRNN` (largest difference 4.8×10^{-5}, from rounding). The plot shows that after 50 steps a factor of 0.5 shrinks the gradient to 8.88×10^{-16} and a factor of 1.5 grows it to 6.38×10^{8}, which is why vanilla RNNs struggle with long-term dependencies.
Inference (RNN):
Loss falls from 1.66 to 0.39 and accuracy rises to 85.42% (training) and 81.11% (validation). The curves are still improving at epoch 30 and validation accuracy never reaches 90%, so the RNN has not fully converged. The slow progress is consistent with all 128 steps passing through a single tanh state.
Inference (LSTM):
Validation accuracy passes 90% at epoch 22 and ends at 92.78% (training 94.70%). There is one instability at epoch 9 (validation loss jumps to about 0.79) that recovers at once. The best validation loss is at the last epoch, so it was still improving slowly.
Inference (GRU):
The GRU converges fastest: validation accuracy passes 90% at epoch 11, the curves flatten after about epoch 12, and it ends at 95.00% (training 96.61%). The validation loss changes little between epochs 25 and 30 (0.1469 to 0.1405), so it is close to converged.
Inference (overfitting and underfitting):
None of the models overfits: validation loss never rises steadily while training loss falls. The generalisation gaps are 4.31 (RNN), 1.92 (LSTM) and 1.61 (GRU) points. The RNN is the closest to underfitting, since its training accuracy is only 85.4% and still rising.
Inference:
LAYING is perfect in all models, and the RNN's worst class is WALKING_UPSTAIRS (20 errors) while the LSTM and GRU's worst is STANDING (11 and 12). The WALKING $\leftrightarrow$ WALKING_UPSTAIRS confusion shrinks from 33 errors (RNN) to 7 (LSTM) and 0 (GRU), so it is model-dependent. SITTING $\leftrightarrow$ STANDING stays at 20, 16 and 16 errors, so it is a limit of the data, and it accounts for all 16 GRU errors.
Inference:
Performance rises from the RNN (84.23% F1) to the LSTM (93.02%) and GRU (95.54%), but size does not: the LSTM is largest (6,006 parameters, 3.04 times the RNN), the GRU has 79% of that (4,758). Training time does not follow size either (RNN 24.43 s, LSTM 22.85 s, GRU 20.28 s), likely because LSTM and GRU use optimised GPU kernels. The GRU gives the best balance of performance, complexity and cost in this run.
Inference:
F1 rises with more context for the GRU (91.39 to 95.54%) and the RNN (74.82 to 84.23%, though it dips to 71.02% at $T = 64$), while the LSTM is nearly flat (92.46, 93.90, 93.02%). Short windows hold less of the repeated pattern, so similar activities are harder to separate. Small dips may be run-to-run variation. Training time rose about 1.4 to 1.5 times from $T = 32$ to $T = 128$.
Inference:
The frozen CNN captures spatial information in each frame (court, hoop, players, ball) as a 1280-dimensional vector, with no notion of time. The LSTM/GRU reads the 10 vectors in order and learns how they change (the player bending, jumping and landing), and only its final hidden state goes to the softmax.
Inference:
Training accuracy reaches 100% by epoch 5, while validation accuracy peaks at 90% and settles at 87.5%, a 12.5-point gap. This is a mild overfitting caused by strong pretrained features on only 160 training videos. One validation video is 2.5 points.
Inference:
TennisSwing is recognised best (12/12) and Basketball worst (7/12). Four Basketball videos were predicted as TennisSwing, all clips of the same recording (g01), which points to a problem with that recording rather than

random noise. With 12 test videos per class, the class-level numbers are unstable.

Inference:

The model is perfect on the test set (100% token and sequence accuracy, both losses 0.0001), so the two accuracies are equal in this run. In general sequence accuracy is about p^L for token accuracy p and length L: 95% token accuracy over 4 tokens gives only about 81% sequence accuracy, and errors compound because the decoder feeds its own output back in.

27. Discussion Questions

1. What is a recurrent neural network?
   Ans: A network that reads a sequence one step at a time and carries a hidden state from step to step, reusing the same weights at every step.
2. What information is represented by the hidden state?
   Ans: A compressed summary of everything seen so far in the sequence, so it acts as the network's memory.
3. Why is sequence order important?
   Ans: The meaning is in the order. Shuffling the 128 sensor readings keeps the values but destroys the walking rhythm.
4. Explain the difference between a feed-forward network and an RNN.
   Ans: A feed-forward network has no memory and treats each input independently. An RNN has a recurrent connection, shares weights across time steps and keeps a state.
5. Explain Backpropagation Through Time.
   Ans: The RNN is unrolled over T steps and the error is propagated back through every step. The gradient of the shared weights is the sum of the contributions from all steps.
6. Why can gradients vanish during BPTT?
   Ans: The gradient is a product of many factors (tanh derivative ≤ 1 and W_h). If they are below 1, the product shrinks exponentially, e.g. $0.5^{50} \approx 9 \times 10^{-16}$.
7. Why can gradients explode during BPTT?
   Ans: If the factors are above 1 the product grows exponentially, e.g. $1.5^{50} \approx 6 \times 10^{8}$. Gradient clipping is the usual fix.
8. What are long-term dependencies?
   Ans: Cases where the output depends on an input many steps earlier, which plain RNNs struggle to learn.
9. What is the role of the LSTM cell state?
   Ans: It is a separate memory line updated by addition ($C_t = f_t C_{t-1} + i_t \tilde{C}_t$), so information and gradients can travel over many steps without shrinking.
10. What are the forget, input and output gates in LSTM?
    Ans: Forget gate: how much old memory to keep. Input gate: how much new candidate to write. Output gate: how much of the memory to expose as h_t.
11. What are the reset and update gates in GRU?
    Ans: Update gate: how much of the old state to keep versus replace with the new candidate. Reset gate: how much of the past to use when computing the candidate.
12. Compare LSTM and GRU.
    Ans: LSTM has 3 gates plus a cell state (6,006 parameters here). GRU has 2 gates and no cell state (4,758). In this experiment GRU was smaller and better (95.54% vs 93.02% F1).
13. Why does GRU generally use a simpler gating mechanism than LSTM?
    Ans: It merges the forget and input gates into one update gate and merges the cell and hidden state, giving fewer parameters and less computation.
14. Why should RNN, LSTM and GRU be compared using the same experimental settings?
    Ans: So that any difference in results comes from the recurrent cell itself and not from the data, optimizer, batch size or split.
15. What does the confusion matrix reveal that accuracy alone does not?
    Ans: Which classes are confused with which. Here it showed that the GRU's 16 errors were all SITTING $\leftrightarrow$ STANDING.
16. Why should macro F1-score be reported for multi-class classification?
    Ans: It gives every class equal weight, so a poorly recognised class lowers the score even if it is small.
17. What is the significance of sequence length?
    Ans: It sets how much temporal context the model sees. Longer sequences gave better F1 for the GRU and RNN here, at higher training time.
18. Why can a CNN be used as a feature extractor for video?
    Ans: A CNN pretrained on ImageNet has learned general visual features, so its pooled output describes each frame compactly without retraining.
19. What type of information is extracted by the CNN?
    Ans: Spatial information within a single frame (objects, shapes, scene layout), with no motion.

20. What type of information is learned by the recurrent network?
    Ans: Temporal information: how the frame features change across the 10 frames.
21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?
    Ans: Frames are ordered in time and the LSTM/GRU expects input of shape $(T, \text{features})$, so each frame's feature vector is one time step.
22. Explain the encoder and decoder in sequence-to-sequence learning.
    Ans: The encoder reads the input sequence and compresses it into a context (its final states). The decoder starts from that context and generates the output one element at a time.
23. What is teacher forcing?
    Ans: During training the decoder is fed the true previous token instead of its own prediction, which speeds up and stabilises training. At test time it must use its own predictions.
24. Differentiate token accuracy and sequence accuracy.
    Ans: Token accuracy = correct tokens / total tokens. Sequence accuracy = fully correct sequences / total sequences, which is stricter (about p^L for token accuracy p).
25. Why must the test set remain untouched during model selection?
    Ans: Otherwise test information leaks into the choices, and the test score becomes an optimistic estimate of real performance.

28. Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.
   Ans:

Table 25: Effect of the number of recurrent units (32 units is the main experiment).

Model	**Units**	**Acc (%)**	**F1 (%)**	**Params**	**Time (s)**
RNN	16	68.33	68.03	790	24.09
RNN	32	84.17	84.23	1974	24.43
RNN	64	79.17	78.82	5878	24.37
LSTM	16	93.33	93.32	2038	19.50
LSTM	32	93.06	93.02	6006	22.85
LSTM	64	95.28	95.25	20086	18.83
GRU	16	94.44	94.44	1670	19.29
GRU	32	95.56	95.54	4758	20.28
GRU	64	95.28	95.24	15542	18.44

   Parameters grow quickly with units, but F1 does not. The LSTM and GRU stay between 93% and 95.6% at every size, so 16 units are already enough. The RNN is best at 32 units (84.23%) and worse at 16 (68.03%) and 64 (78.82%). Training time did not increase with units, so those differences are mostly run-to-run variation.
2. Compare GRU and LSTM using the same number of recurrent units.
   Ans:

Table 26: GRU compared with LSTM at equal units.

Units	**GRU / LSTM params**	**F1 (GRU − LSTM)**	**GRU / LSTM time**
16	82%	+1.13	0.99
32	79%	+2.52	0.89
64	77%	−0.01	0.98

   At equal units the GRU has 77–82% of the LSTM's parameters and is equal or better in F1 (up to +2.52 points), with the same or slightly shorter training time. The GRU matches the LSTM with fewer parameters.
3. Add a second recurrent layer and investigate whether performance improves.
   Ans:

Table 27: One versus two stacked recurrent layers (32 units each).

Model	Layers	Acc (%)	F1 (%)	Params	Time (s)
RNN	1	84.17	84.23	1974	24.43
RNN	2	81.94	82.17	4054	38.54
LSTM	1	93.06	93.02	6006	22.85
LSTM	2	95.00	94.99	14326	28.70
GRU	1	95.56	95.54	4758	20.28
GRU	2	95.56	95.54	11094	27.27

A second layer helped only the LSTM (+1.97 F1 points). It left the GRU unchanged and made the RNN worse (−2.06), while costing 2.1–2.4 times the parameters and 1.3–1.6 times the time. For this small task one layer is enough.

4. Compare a bidirectional LSTM with a unidirectional LSTM.
   Ans:

Table 28: Unidirectional LSTM versus bidirectional LSTM (32 units).

Quantity	LSTM	BiLSTM
Accuracy (%)	93.06	94.44
Macro F1 (%)	93.02	94.44
Parameters	6006	11894
Training time (s)	22.85	29.39

Reading the window in both directions improved F1 by 1.42 points (about 5 test windows) but nearly doubled the parameters and increased the time by 1.29 times. The gain is small and still below the GRU (95.54%).

5. Change the sequence length and study the effect on computational cost.
   Ans:

Table 29: Training time (s) for different sequence lengths.

Sequence Length	RNN	LSTM	GRU
32	17.05	15.17	14.85
64	18.21	16.55	16.86
128	24.43	22.85	20.28

Training time rises with sequence length but less than proportionally: a four-times longer sequence (32 to 128) increased the time by only 1.43 times (RNN), 1.51 times (LSTM) and 1.37 times (GRU), while the parameter count stays the same. Part of each epoch is fixed overhead that does not depend on T, which is a likely reason.

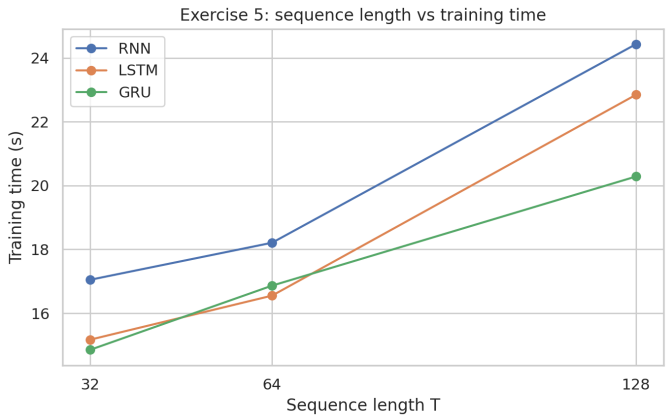


Figure 17: Exercise 5: sequence length versus training time.

6. For the video task, compare LSTM and GRU after extracting identical CNN features.
   Ans:

Table 30: CNN–LSTM versus CNN–GRU on the same MobileNetV2 features.

Quantity	CNN–LSTM	CNN–GRU
Best validation accuracy (%)	90.00	90.00
Test accuracy (%)	85.42	91.67
Test macro F1 (%)	84.84	91.59
Parameters	168196	126276
Training time (s)	7.77	6.56

Both tie on validation accuracy, but the GRU is better on the test set (91.67% versus 85.42%, 3 more videos out of 48) with 25% fewer parameters and slightly shorter training. With 48 test videos and one run, this is only an indication that the GRU is at least as good as the LSTM here.

7. Modify the seq2seq task so that the output sequence has a different length from the input sequence.
Ans: The input has 6 symbols and the output is the last 3 symbols in reverse order (length 3, not 6), with the same encoder–decoder as before.

Table 31: Test examples (input length 6, output length 3).

Sample	Input	Expected	Predicted
1	[7, 6, 5, 3, 5, 3]	[3, 5, 3]	[3, 5, 3]
2	[9, 1, 7, 3, 1, 3]	[3, 1, 3]	[3, 1, 3]
3	[4, 9, 8, 9, 9, 4]	[4, 9, 9]	[4, 9, 9]
4	[5, 6, 5, 4, 1, 3]	[3, 1, 4]	[3, 1, 4]
5	[6, 3, 4, 3, 4, 4]	[4, 4, 3]	[4, 4, 3]

Token accuracy was 100.00%, sequence accuracy 100.00%, final training loss 0.0000 and final validation loss 0.0001. It works with different lengths because the decoder simply runs for as many steps as the target needs (3), independent of the input length (6).

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

$$\boxed{\text{Sequential Data} \rightarrow \text{Preprocessing} \rightarrow \text{RNN/LSTM/GRU} \rightarrow \text{Evaluation}}$$

and the video-understanding pipeline:

$$\boxed{\text{Video} \rightarrow \text{Frames} \rightarrow \text{CNN Features} \rightarrow \text{LSTM/GRU} \rightarrow \text{Action Prediction}}$$

They should also understand the encoder–decoder formulation:

$$\boxed{\text{Input Sequence} \rightarrow \text{Encoder} \rightarrow \text{Context} \rightarrow \text{Decoder} \rightarrow \text{Output Sequence}}$$

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student's own execution and should not be assumed in advance. **Conclusion:**

All three pipelines were implemented and evaluated on the student's own runs.

- **Sequence learning (UCI HAR):** on the 6-class activity task the GRU gave the best test result (95.56% accuracy, 95.54% macro F1, 4,758 parameters), followed by the LSTM (93.06%, 6,006 parameters) and the RNN (84.17%, 1,974 parameters). The gated models handled the 128-step sequences much better than the plain RNN, and the remaining errors were mostly SITTING $\leftrightarrow$ STANDING.
- **Video understanding (UCF101 subset):** frozen MobileNetV2 features (10×1280 per video) with an LSTM gave 85.42% test accuracy on 4 action classes (the model chosen on validation). The GRU on the same features gave 91.67%, and the errors came mainly from Basketball clips predicted as TennisSwing.
- **Sequence-to-sequence:** the encoder–decoder learned the reversal task perfectly (100% token and sequence accuracy), including a variant with a different output length.

Overall, gated recurrent units are better suited than a vanilla RNN for sequences, and pretrained CNN features combined with a recurrent layer provide a simple and effective way to classify videos. The numerical values above come from the student's own execution. With small test sets and single runs, differences of a few points should be read as indications rather than firm rankings.

30. Source Code

`GitHub:` https://github.com/Kanishkarane/RNN-LSTM-GRU

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
3. Kyunghyun Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP, 2014.
4. Anguita et al., "A Public Domain Dataset for Human Activity Recognition Using Smartphones," ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, "UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild," 2012.
6. TensorFlow Documentation: `https://www.tensorflow.org`
7. Keras Documentation: `https://keras.io`